Top-Down Parsing

1. Given the grammar rule for an if-statement:

If-stmt → if (*exp*) *statement*
| if (*exp*) *statement* else *statement*

write pseudo-code to parse this grammar by recursive descent

2. Consider the grammar

expr → expr addop term|term

addop → +|-

term → term mulop factor | factor

mulop →*

factor →(expr) | number

a) Remove the left recursion
b) Construct First and Follow sets for the non terminal of the resulting grammar
Write out each choice separately in order:
c) Construct LL(1) parsing table for the resulting grammar
d) Show the actions of the corresponding LL(1) parser, given the input string 3+4

3. Consider the following grammar

Statement → if-stmt | other

If-stmt → if (exp) *statement*
| if (exp) *statement* else *statement*

Exp → 0 | 1

a) Left factor this grammar
b) Construct First and Follow sets for the non terminal of the resulting grammar
c) Construct the LL(1) parsing table for the resulting
d) Show the action of corresponding LL(1) parser given the input string **If (0) if (1) other else other**

4. Consider the following grammar

Stmt-sequence→stmt; stmt-sequence | stmt

Stmt→s

a) Left factor this grammar
b) Construct First and Follow sets for the non terminal of the resulting grammar
c) Construct the LL(1) parsing table for the resulting
d) Show the action of corresponding LL(1) parser given the input string **s; s**

5. Given the grammar

exp → exp addop term|term

addop → + |-

term → term mulop factor | factor

mulop →*

factor →(exp)| number

Write pseudo-code to parse this grammar by recursive descent

6. Consider the following grammar

S→(S) S|ε

a) Construct First and Follow sets for the non terminals of the grammar
b) Construct the LL(1) parsing table for the resulting
c) Show the action of corresponding LL(1) parser given the input string ( )

7. Left factor the following grammar:

exp → term+exp |term